



CSG-based ML-supported 3D translation of sketches into game assets for game designers

Yiming Chen¹ · Yihang Liu¹ · Gizem Kayar-Ceylan¹

Accepted: 8 December 2024 / Published online: 5 January 2025

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

This research project aims to develop a new technique for translating 2D sketches into 3D assets like building models using machine learning and constructive solid geometry (CSG) algorithms. The ability to quickly create 3D building models from simple freehand sketches has widespread applications in fields like architecture, construction, and interactive design. The key innovation is the use of CSG principles to understand geometric elements from sketches, extrude them into 3D, and combine simple shapes to create complex structures. This allows the creation of building models with walls, windows, doors, and other features that closely match the original sketch, without requiring extensive 3D modeling expertise. Specialized sketch recognition methods including machine learning techniques for edge detection and point decimation are used to identify lines, arcs, and intersections, and transform them into the necessary CSG primitives. We plan to develop a user-friendly web interface where users can sketch freehand, and see a live 3D model preview generated in real-time by reconstructing surfaces from the identified geometric features. This integrated sketching and 3D modeling environment provides an intuitive workflow, harnessing CSG techniques to generate complex forms from sparse 2D input. Overall, this research aims to improve sketch-based 3D modeling methods and enable more users to create 3D content with ease. The researchers believe this work has the potential to significantly enhance productivity in construction planning, enable faster design iterations, and increase architectural creativity.

Keywords 3D translation of sketches · Constructive solid geometry · Game engine integration

1 Introduction

The ability to quickly and intuitively translate 2D sketches into 3D assets, e.g., building models, has widespread applications in fields such as architecture, construction, video game design, and interactive media. Architects, designers, and artists often begin the creative process by hand-drawing rough conceptual sketches, which could be tremendously valuable if they could be instantly transformed into 3D representations [1–3]. This could aid in the visualization and iteration of building designs, help translate abstract blueprints into physical visualizations, and empower those

without extensive 3D modeling expertise to create virtual environments and assets.

However, the technical challenge of reliably reconstructing 3D geometry from simple 2D sketches remains an open problem. While humans can easily interpret the meaning and structure of basic sketch elements like walls, windows, and doors, computer vision and geometry processing algorithms continue to struggle with this task [4, 5]. Existing sketch-to-3D techniques often rely on standardized geometric drawings, large training datasets, or can only handle simple shapes, limiting their real-world applicability [6, 7].

It is worth noting that many of the commercially available Large Language Models (LLMs) have begun to support the generation of 3D images and videos [8]. However, most of them only produce 2D images or videos with 3D concepts [9]. There is no model that works well for generating 3D objects that can be applied to games or other 3D designs [6]. For sketches, there are models that can generate images from sketches; however, most of the models are not sensitive to the vertex position information in the sketches [6, 9, 10].

✉ Yiming Chen
yc4346@nyu.edu

Yihang Liu
yl7838@nyu.edu

Gizem Kayar-Ceylan
gk2409@nyu.edu

¹ New York University, New York, NY 10012, USA

As a result, the generated images are not good enough for the design needs.

This research aims to develop a new approach that can robustly translate freehand sketches of any style into 3D building models maintaining significant vertex or edge features. By leveraging specialized sketch recognition methods and a unified sketching-modeling environment, the proposed system seeks to enable intuitive 3D content creation from sparse 2D input. The key innovations include the use of Constructive Solid Geometry (CSG) to automatically extrude and combine basic shapes into complex structures, and the integration of this 3D reconstruction pipeline with a user-friendly web interface for real-time sketch-based modeling. As shown in Fig. 1, the user interface for our application includes a sketch area on the left side for users to input 2D sketches, coupled with a real-time 3D preview window that displays the transformation into 3D models. It features a tool palette for direct sketch manipulation like rectangle or circle drawers. Essential action buttons for screenshotting and exporting are present, along with navigation controls to explore the model with different selection of textures. This setup aims to facilitate intuitive 3D content creation from simple sketches, enhancing accessibility and user engagement.

Overall, this work has the potential to significantly enhance productivity, enable faster iteration, and increase accessibility in a variety of 3D design applications. The remainder of this paper is organized as follows: Sect. 2 presents related work in sketch analysis, Constructive Solid Geometry, texture generation, and 2D-to-3D generation techniques. Section 3 details our methodology, including sketch recognition and geometric analysis, 3D reconstruction with CSG operations, diffusion-based texture generation, and web deployment implementation. Section 4 demonstrates our experimental results with various sketch inputs and their

corresponding 3D model outputs. Finally, Sect. 5 discusses future work and potential improvements to our system.

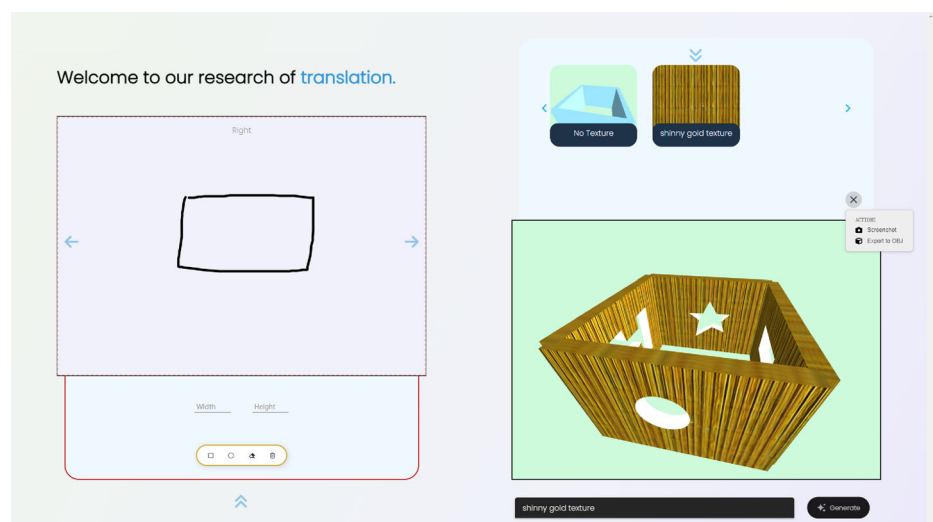
2 Related work

Sketch analysis

Most of the sketches contained in existing datasets of hand-drawn sketches are single-object [11–13], while datasets with sketches covering multiple objects are mostly generated from photos [6, 14, 15]. Training deep neural networks (DNNs) to recognize sketches with multiple objects is challenging due to the unique characteristics of sketches, such as their abstract nature and lack of color and texture information. Recent studies on object detection in computer vision have shown that DNNs based on bounding boxes, like those implemented in YOLO, can effectively distinguish multiple objects in photos [16, 17]. However, these DNNs designed for photos do not perform well on sketches [18]. Unlike photographs, sketches are sparse and abstract, which complicates object recognition [18–21].

Sketches can be represented in multiple ways: as static pixel images, dynamic stroke sequences, and geometric graphs [21]. The state-of-the-art model for sketch recognition is Sketch-a-Net, a DNN-based model that achieves a 74.90% accuracy on the TU-Berlin dataset [21]. Despite its success, there has been relatively little research on multi-object sketch analysis in recent years, likely due to the lack of new datasets. Most existing models can only study single-target sketches and output a single result without providing positional information [6, 10].

Fig. 1 3D reconstruction pipeline



For shape recognition from sketches, vertex-based polygon analysis combines both efficiency and accuracy and also returns positional information based on vertices [22–25]. This approach addresses the limitations of current DNN-based models by offering a more detailed understanding of the sketch structure.

Constructive solid geometry

Constructive Solid Geometry (CSG) is a modeling technique used in computer graphics that builds complex surfaces or objects by using Boolean operations to combine simpler ones [26]. CSG is advantageous because of its high scalability, intuitive modeling process, and efficient data management of simple primitive models, resulting in fast rendering and easy retrieval of complex shapes. CSG models can achieve high accuracy and are highly scalable, accurately representing complex geometries with simple primitives such as cubes, spheres, and cylinders. The intuitive process of creating models with CSG - using operations such as union, intersection, and difference - makes it accessible to users without extensive 3D modeling expertise. Moreover, CSG's use of simple primitive models allows for efficient data management and quick rendering, particularly useful in applications requiring real-time performance, such as video game engines [26].

In video game development, CSG is integrated into game engines to generate complex 3D environments and assets, enhancing visual quality and interactivity. While traditional CSG techniques have been widely used, recent machine-learning approaches have brought new perspectives to 3D model generation. These ML-based methods, such as advanced edge detection and point decimation techniques [27], provide innovative ways to transform sketches into 3D models. Similarly, real-time web-based interfaces have demonstrated the potential of integrating deep learning with traditional modeling approaches, enabling instant sketch-to-3D conversion while maintaining high visual quality [28]. These advancements suggest potential opportunities for refreshing traditional CSG techniques with modern ML capabilities.

Texture generation

Building on the foundations laid by previous research in generative models, the "Denoising Diffusion Probabilistic Models" by Ho et al [29], presents a comprehensive framework for image synthesis that draws from both diffusion probabilistic models and denoising score matching techniques. This approach, rooted in the principles of nonequilibrium thermodynamics, innovatively utilizes variational inference to reverse a diffusion process characterized by incremental Gaussian noise applications, thereby generating high-fidelity images.

Significant progress in high-resolution image synthesis has been achieved through various approaches. Generative Adversarial Networks (GANs), introduced by Goodfellow et al., have been pivotal in generating high-quality images, though they often encounter stability challenges [30]. Variational Autoencoders (VAEs) and flow-based models, as developed by Kingma and Welling, provide robust frameworks but sometimes at the expense of image quality [31]. Recently, diffusion models, particularly those advanced by Ho et al. and Dhariwal and Nichol [29, 32, 33], have shown promise in balancing image quality with training stability by utilizing reweighted variational objectives. These methods underpin our research, where we extend the capability of Latent Diffusion Models (LDMs) to efficiently produce detailed high-resolution images in a reduced computational space, advancing the state-of-the-art in the field.

2D-to-3D generation based on sketches

Recent advancements in deep learning and computer vision have led to significant progress in 2D-to-3D generation based on sketches. One of the primary approaches in this domain is image-to-image translation models, such as pix2pix3D, which focus on maintaining 3D realism when viewing objects from different angles [28]. These models typically utilize Generative Adversarial Networks (GANs) to generate high-quality 3D representations from 2D inputs [28]. While some approaches focus on specific domains such as tree sketches [20], these methods still require synthetic sketches rather than freehand sketches as input.

In order to overcome these constraints, research has looked at approaches other than deep learning. Tanaka et al., for example, proposed two complementary methods for automatically transforming 2D sketches into 3D mechanical models. In their earlier work [34], they introduced the "Sketch Feature-Based Conversion Method (SFBCM)," which defines "sketch faces" and "sketch features" to progressively reconstruct intricate shapes by detecting and combining fundamental geometric elements such as cuboids, cylinders, and holes. This method relies on systematic algorithms with minimal dependence on machine learning, making it particularly suited for mechanical designs with clear and structured inputs. Their subsequent work expanded on this by stressing the importance of human perception and using designers' inclination for symmetrical and isometric drawings to streamline the conversion process [35]. This improvement addressed problems including confusing intersections and hidden elements while broadening the method's applicability to a wider variety of structured sketches. Both methods, however, are still limited by the requirement for exact and well-defined inputs and struggle to generalize to rough or ambiguous sketches. Future developments will focus on improving the handling of less organized sketches

for broader applicability and broadening the dictionary of sketch attributes.

Another direction is to generate 3D models in the form of point clouds like the work by Jiayun Wang et al [36]. This method exploits the inherent spatial information in point clouds to generate more accurate 3D shapes [36]. Despite these advances, a significant challenge remains the lack of comprehensive training datasets, which limits the ability of these models to generalize to different sketch styles and complexities. In addition, many existing models are limited to handling simple shapes and often struggle with more complex, multi-object sketches.

Recent research has sought to address these limitations by integrating multi-view learning and shape-from-shading techniques. For instance, multi-view learning models synthesize multiple 2D views of a sketch to infer the 3D structure, improving the robustness and accuracy of the generated models. Shape-from-shading techniques, on the other hand, utilize shading information in sketches to better understand the depth and curvature of objects, leading to more realistic 3D reconstructions [37]. Furthermore, hybrid models combining CSG principles with deep learning have shown promise in generating more precise and geometrically accurate 3D models from sketches [27]. These advancements are crucial for developing robust 2D-to-3D generation systems that can handle a wide range of sketch inputs and produce high-quality 3D models suitable for various applications. Overall, not limited to sketches as input, 2D-to-3D is still a problem without well-established generative model solutions.

Feature detection and procedural modeling from sketches

Sketch-based modeling (SBM) and feature detection have long been crucial for bridging the gap between 2D sketches and 3D models. Early work in sketch-based modeling (SBM) aimed to accelerate the production of 3D models from sketches by focusing on specific feature detection methodologies [38]. In single-view 2D sketches, Plumed et al. presented a novel framework for identifying design and manufacturing elements such as stairs and pockets. Their reverse-design-history approach analyzed 2D line-drawings to build CSG feature trees, which allowed for direct transformation of sketches into CAM-ready models.

Although conventional techniques, such as FlexiCAD, used preset geometric patterns to identify non-interacting features, they were frequently limited by axis-aligned constraints. In order to improve geometric interpretation and scalability and facilitate automatic feature recognition and smooth 3D reconstruction, recent developments have concentrated on combining machine learning with CSG. Despite these advancements, processing abstract and multi-feature

sketches still presents difficulties because of their sparseness and spatial uncertainties. Techniques that integrate graph-based algorithms with vertex analysis have been developed to better preserve the structural function of sketches, leading to more accurate and user-friendly 3D modeling solutions.

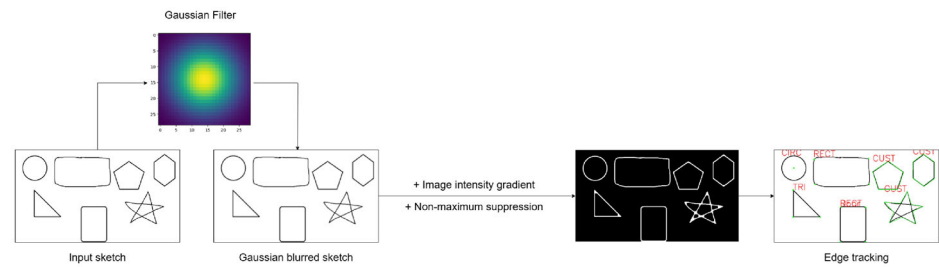
Because of the intrinsic constraints of current algorithms in understanding geometric and semantic complications, processing slanted features, non-parallel edges, and sketches with significant abstraction is still difficult despite recent developments [39]. Since typical systems have trouble extrapolating features that depart from orthogonal planes or exhibit variable cross-sections from sketch data that is unclear or fragmentary, these features frequently require sophisticated techniques for proper reconstruction. Furthermore, abstract sketches, which are frequently employed throughout the conceptual design process, bring additional issues such as missing dimensions, ambiguous proportions, and imperfect representations of elements. To tackle these problems, robust algorithms that allow for design context, infer missing data, and maintain design intent across the modeling process are necessary. Emerging methods such as auxiliary reference systems and indirect cues, including symmetry planes, skewed datums, and hierarchical relationships between features, have shown potential in tackling these complexities. These techniques leverage structural and spatial information to construct more accurate feature hierarchies, enabling better alignment of abstract sketches with detailed 3D representations. Furthermore, advancements in procedural modeling approaches and automated feature extraction algorithms have enhanced interoperability between different design tools, simplifying workflows and improving efficiency. These developments hold promise for expanding the accessibility of 3D modeling, empowering non-expert users, and facilitating more intuitive design processes. Future research continues to focus on refining these systems by incorporating advanced machine learning techniques for adaptive feature recognition and exploring interactive tools that can seamlessly bridge conceptual sketches and detailed 3D models.

3 Method

3.1 Sketch recognition and geometric analysis

The foundation of the sketch-to-3D reconstruction pipeline is the robust recognition and analysis of the input sketch geometry. To achieve this, we employed a multi-step computer vision approach leveraging both traditional image processing techniques and machine learning based shape detection.

We trained a CNN-based shape classifier on the Hand-drawn Shapes (HDS) Dataset, which contains 27,292 hand-drawn shapes with 4 categories (Ellipse, Rectangle, Triangle,

Fig. 2 Image filter processing

and Other) [13]. Following the dataset's instruction, we used a CNN architecture consisting of 6 convolutional layers (with 32, 64, and 128 filters), batch normalization, and maxpooling layers, followed by dense layers for final classification. Our training set has 21,386 samples, the validation set has 3,983 samples and the test set includes 1,923 samples. We used a 128 batch size. The optimizer we used is Adam with an initial learning rate of 0.001. The loss function is categorical cross-entropy. The model is trained on an NVIDIA GeForce RTX 3060 Laptop GPU for 100 epochs. The model achieved 95.74% accuracy on the test set. According to the founder of the dataset, a CNN-based model can achieve 99.95% accuracy on the training set [13].

However, the high accuracy of the model on the dataset may overfit the dataset. We found that the model performs poorly on hand-drawn shapes with openings. In addition, it cannot detect multiple shapes or other polygons. For practical purposes, we wanted to be able to detect multiple shapes in a single image input, as well as handle some custom polygons. To solve this problem, we found that traditional computer vision techniques are up to the task and very efficient.

The process begins by loading the input sketch image and applying several preprocessing steps to enhance the sketch features and prepare the data for analysis. First, a Gaussian blur filter is used to smooth the image and reduce noise, helping to clean up the hand-drawn sketch lines. The Gaussian blur filter works by averaging the pixel values in a neighborhood around each pixel in the image. The averaging process is weighted by a Gaussian function, which is a bell-shaped curve that decreases in value as you move away from the center pixel. This results in pixels closer to the center of the neighborhood having a greater influence on the average than pixels farther away, creating a smoothing effect that preserves edges while reducing high-frequency noise. By applying the Gaussian blur, fine details and minor imperfections in the sketch are smoothed out, making the primary structures more prominent and easier to detect. This preprocessing step is crucial for subsequent stages, as it enhances the clarity of the sketch, allowing for more accurate feature extraction and geometric analysis. The degree of blurring can be controlled by adjusting the standard deviation of the Gaussian function, which determines the width of the bell curve. A larger standard deviation results in more extensive blurring, while

a smaller standard deviation focuses on reducing only minor noise without significantly altering the overall structure of the sketch.

Next, Canny edge detection is performed to identify the key edges and contours in the sketch like shown in Fig. 2. The Canny edge detection algorithm is a multi-stage process that involves several steps. These include noise reduction using a Gaussian filter, gradient calculation, non-maximum suppression, and edge tracking by hysteresis.

1. Gaussian filter: It reduces noise in a manner analogous to the initial Gaussian blur step. This prepares the image for edge detection by smoothing out minor variations that could otherwise lead to false edge detection.
2. Image intensity gradient: It is calculated using finite difference approximations. This step involves computing the gradient of the image intensity at each pixel, identifying areas of rapid intensity change, which correspond to edges. The gradients are calculated in both horizontal and vertical directions to determine the strength and direction of the edges.
3. Non-maximum suppression: It is then applied to the edges to refine the edge map. This technique involves examining the gradient magnitude of each pixel and retaining only the local maxima, where the gradient magnitude is the highest. By doing this, non-maximum suppression helps in eliminating extraneous details and reducing the thickness of the edges, resulting in a cleaner, more precise edge representation.
4. Edge tracking by hysteresis: It employs two thresholds to identify strong and weak edges. Strong edges are those with gradient magnitudes above the higher threshold and are considered definitive edges. Weak edges, which have gradient magnitudes between the two thresholds, are included in the edge map only if they are connected to strong edges. This connectivity check helps in preserving the continuity of edges and discarding isolated weak edges that are likely to be noise.

Therefore, Canny edge detection algorithm produces a binary image that highlights the significant edges and contours in the sketch, which are crucial for further processing and 3D reconstruction. This method ensures that the detected edges

are both accurate and meaningful, providing a robust foundation for subsequent modeling tasks.

A morphological closing operation is applied to refine the edge information further to close any gaps in the detected edges. With the preprocessed sketch data, the next step is to detect and classify the various geometric shapes present in the drawing. This is accomplished by iterating through the detected contours and approximating each contour with a polygon using the `cv2.approxPolyDP()` function. This function takes a contour as input and approximates it with a polygon. The function works by iteratively reducing the number of vertices in the polygon, while ensuring that the maximum distance between the original contour and the approximated polygon is less than a specified epsilon value. This allows the function to find the minimal number of vertices required to accurately represent the original contour shape. Specifically, the algorithm checks the following criteria to identify different shape types:

- Triangles: Identified by 3 vertices forming a convex polygon
- Rectangles: Identified by 4 vertices forming a convex polygon with specific edge angle properties
- Circles: Identified by closed contours that satisfy circularity criteria based on the bounding box and area

For each detected shape, the vertex coordinates are extracted and stored, along with metadata like the bounding box dimensions. This information provides the essential geometric data needed for the subsequent 3D reconstruction stage. Additionally, the algorithm attempts to identify custom, irregularly shaped objects that don't fit the predefined shape categories. For these cases, the vertex coordinates of the approximated polygon are still captured and stored, enabling more complex geometry to be incorporated into the final 3D model.

An important aspect of the sketch recognition process is the classification of doors and windows. Any detected shapes with their bottom edge below 5% of the image height are classified as potential doors and labeled accordingly in the output. This contextual information is critical for properly interpreting the architectural intent behind the sketch.

The extracted geometric information from the sketch analysis is written to a specified output text file. The format of the output is designed to be structured and easily parsable for the subsequent 3D reconstruction steps. The first line of the output contains the width and height of the canvas, which provides the necessary context for interpreting the coordinate values. The subsequent lines are grouped by shape type, starting with a single-character identifier (C for circles, T for triangles, R for rectangles, U for custom/irregular shapes). For each detected shape, the relevant vertex coordinates and other metadata (e.g., radius for circles, bounding box dimensions for rectangles) are listed on a new line. By

processing the input sketches one by one and appending the output to the same text file, the system is able to generate a comprehensive representation of the entire building's geometry, with each wall's elements clearly labeled and organized. This structured output format ensures that the downstream 3D reconstruction module can reliably interpret the extracted sketch data and generate an accurate 3D model.

Algorithm 1: Sketch Reading and Shape Detection

```

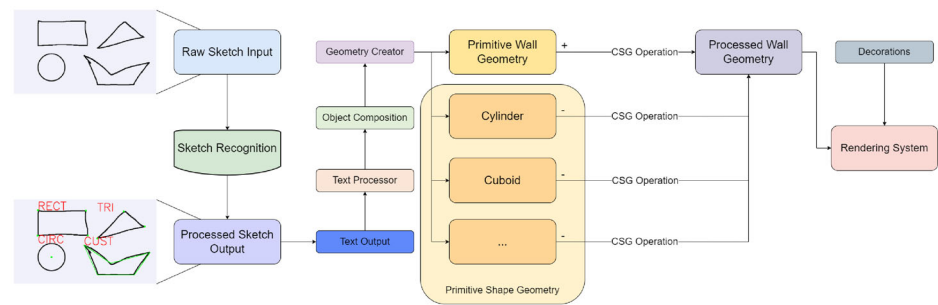
Data: Input sketch image
Result: Textual data of detected shape
Load the image and preprocess;
image  $\leftarrow$  load_image();
blurred_image  $\leftarrow$  apply_gaussian_blur(image);
gray_image  $\leftarrow$  convert_to_grayscale(blurred_image);
edges  $\leftarrow$  apply_canny_edge_detection(gray_image);
closed_edges  $\leftarrow$  apply_morphological_closing(edges);
contours  $\leftarrow$  find_contours(closed_edges);
Initialize lists for shape info;
circle_info, triangle_info, rectangle_info, custom_object_info
 $\leftarrow$  [], [], [], [];
for each contour in contours do
    Get the polygonized contour;
    approx  $\leftarrow$  approximate_contour(contour);
    Classify the shape;
    if is_triangle(approx) then
        store_triangle_info(triangle_info, approx);
    else
        if is_rectangle(approx) then
            store_rectangle_info(rectangle_info, contour);
        else
            if is_circle(contour) then
                store_circle_info(circle_info, contour);
            else
                store_custom_object_info(custom_object_info,
                    approx);
Write output and save image;
write_output_text_file(width, height, circle_info, triangle_info,
    rectangle_info, custom_object_info);
save_output_image(image);

```

We may notice that Algorithm 1 processes an input sketch image to detect and classify various shapes, producing textual data as output. It begins by loading and preprocessing the image, including applying Gaussian blur to reduce noise, converting it to grayscale, and performing Canny edge detection to identify edges. Morphological closing is then applied to close gaps in the edges, followed by contour detection.

For each detected contour, the algorithm approximates the contour to a polygon and classifies it as a triangle, rectangle, circle, or custom object based on its geometric properties. The classified shape information is stored in corresponding lists. Finally, the algorithm writes the detected shape information to a text file and saves the processed image, completing the shape detection process.

Fig. 3 3D reconstruction pipeline



3.2 3D reconstruction with CSG

The key to converting the 2D sketches into 3D models is the use of Constructive Solid Geometry (CSG) operations as shown in Fig. 3. To begin the 3D reconstruction from the text output from the sketch analysis, we have a text processor. The text processor parses the text, recognizes the shape type of each object, and stores each object's data in the object composition as an array. The object composition is an array that contains a nested composite data type for each wall. Each nested composite data type contains arrays of wall sizes and object data. This design allows quick access to the data for each wall and each object.

The geometry creator is a function that turns walls and objects into geometry objects. It first creates a wall and then the shape objects on this wall. The geometry for walls derives from the geometric transformations of a cube. These are the primitive wall geometry objects. For each shape object on the walls, the geometry creator uses the shape data, such as vertices, to draft a planar shape and then make a column by panning. We call these geometry objects from the shape objects Primitive Shape Geometry (PSG) objects. PSG objects are simple, which is favorable for the following CSG operation. Also, the simplicity of PSG helps achieve the quick construction necessary for real-time reconstruction.

The process of creating openings in walls, as depicted in Fig. 4, involves three key steps. First, the opening shape is marked on the primitive wall geometry, as shown on the left. Next, an extrusion primitive is created based on the marked opening, which is illustrated in the middle section of the figure. Finally, a CSG subtraction operation is performed to remove the extruded shape from the wall geometry, resulting in a clean opening, as seen on the right.

For each PSG object, a CSG operation is conducted to subtract the PSG object from the corresponding primitive wall geometry object. After finishing all CSG operations, we have the processed wall geometry objects with voids corresponding to the shapes in the sketches. Through the data stored by the object composition, further windows or other decoration objects can be generated and settled on the matching voids. For instance, we can use a series of CSG operations with primitive 3D geometry objects to create decoration objects.

Likewise, creating "steps" or similar features involves applying the CSG addition operation, which is straightforward and has no technical constraints.

The processed wall geometry objects are then transferred to the rendering system with decorations, including the generated textures and decoration objects. All geometry objects are turned into mesh objects in the rendering system. According to the workflow we have presented, most of the 3D object operations are conducted on geometry objects. It is also an important feature aiding the efficiency needed for real-time reconstruction.

3.2.1 Blind holes

In addition to through-holes, the system now supports the creation of blind holes (non-through holes) within the walls, enabling more intricate architectural designs. Blind holes are partial indentations into the walls rather than complete cut-throughs. This feature enhances the flexibility of the system to create niches, recesses, or other decorative architectural features.

As shown in Fig. 5, the depth of each blind hole is explicitly defined by the user through a progress bar in the user interface. This interactive control enables precise adjustments to the depth of the cut, offering users a high degree of customization for their designs. The progress bar dynamically updates the depth parameter, which is applied during the modeling process.

The integration of user-defined depth values ensures consistency and accuracy in the modeling process. During the extrusion step, the geometry creator positions the extrusion primitive at the specified depth within the wall thickness, causing the CSG subtraction operation to reach the intended area. By aligning the primitive's position with the user-defined depth, the system guarantees the creation of blind holes while preserving the structural integrity of the remaining wall.

This workflow enables users to preview depth adjustments in real time, further improving the system's interactivity and usability. The implementation of blind holes complements the existing through-hole functionality, expanding the system's capability to handle a broader range of architectural

Fig. 4 Wall opening creation process

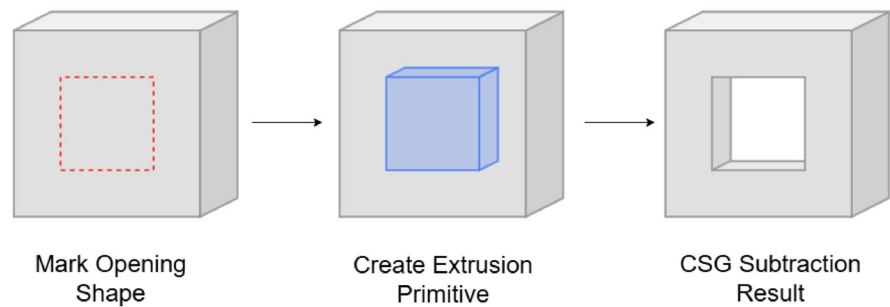
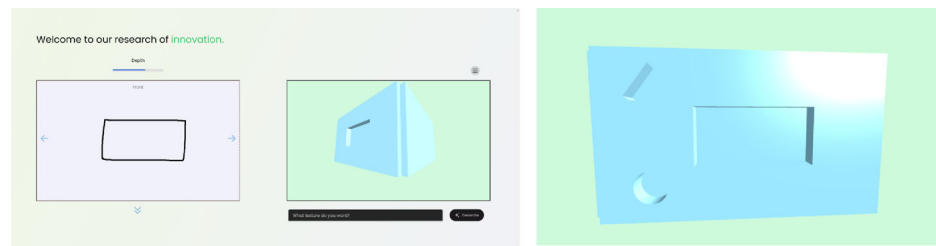


Fig. 5 Blind hole creation and example



features. By providing users with control over the depth of each blind hole, the system empowers them to create customized and detailed designs suitable for various applications, including architectural visualization, game design, and interior modeling.

3.3 Diffusion-based texture generation for 3D models

We have integrated a diffusion-based texture generation approach to automatically create textures for the 3D building models reconstructed from the sketch input. This technique leverages the impressive capabilities of diffusion models, a class of deep learning models that have shown remarkable success in generating high-quality, diverse images from textual descriptions. As shown in Fig. 6, the core of the texture generation component is the Stable Diffusion pipeline, which is a pre-trained denoising diffusion probabilistic model made available by Hugging Face [40].

Diffusion models work by learning to gradually transform random noise into realistic images that match a given textual prompt. This is achieved through a process of iterative refinement, where the model learns to predict the step-by-step changes required to transform the initial noise into the desired output. In the context of this sketch-to-3D reconstruction research, the texture generation process is invoked whenever a 3D building model needs to be textured. The user provides a textual prompt that describes the desired style or theme for the texture, such as "modern office texture" or "medieval wooden texture". This prompt is then fed into the Stable Diffusion pipeline, which generates a high-quality, photorealistic image that matches the given description. The

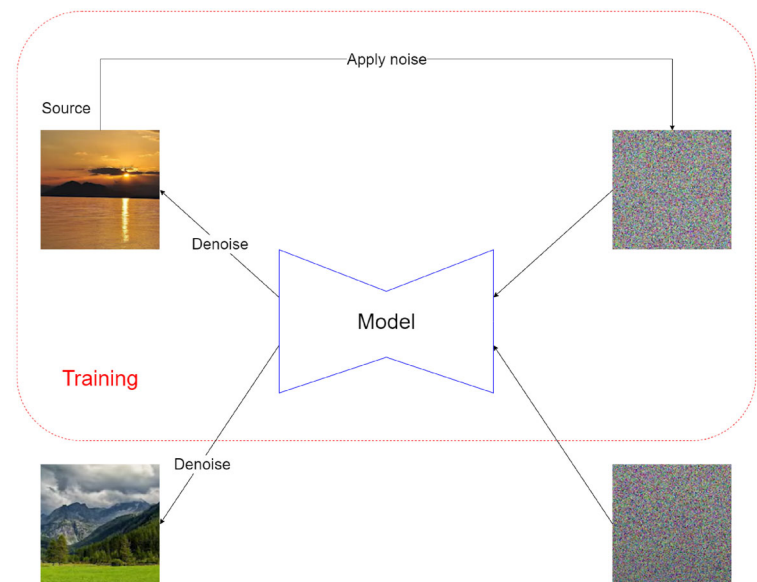
architecture of the Stable Diffusion model can be broadly divided into two main components:

- Encoder: The encoder network takes the input text prompt and encodes it into a compact, semantic representation. This representation captures the high-level features and attributes of the desired texture, such as the materials, colors, and architectural elements.
- Denoiser: The denoiser network is responsible for progressively refining a random noise input into the final, high-quality texture image. It does this by learning to predict the step-by-step transformations required to gradually remove the noise and shape the image to match the encoded text prompt.

The diffusion process works by iteratively applying a series of learned noise-adding and noise-removing operations to the input. In the initial stages, the input is heavily corrupted with noise, and the model learns to predict how to gradually remove this noise to reveal the underlying image structure. As the process continues, the model becomes increasingly capable of generating sharper, more detailed textures that closely align with the given prompt.

One of the key advantages of using a diffusion-based approach for texture generation is its flexibility and generalization capability. Since the model is trained on a large and diverse dataset of images and text pairs, it can generate textures for a wide range of architectural styles and materials, beyond what might be feasible with traditional texture synthesis or manual creation methods. In the context of the sketch-to-3D reconstruction pipeline, the generated textures are seamlessly integrated with the 3D building models, providing a visually appealing and realistic final result. This

Fig. 6 Training and application of a denoising diffusion probabilistic model: a visual representation of noise addition and reduction in image processing



diffusion-based texture generation component significantly enhances the overall quality and fidelity of the 3D models, making them more suitable for a variety of applications, such as architectural visualization, interactive design, and video game asset creation.

3.4 Web-deployed sketch-based 3D modeling system

3.4.1 Web graphics library (WebGL) and Three.js

We used WebGL [41] and the Three.js [42] library to handle the 3D rendering and visualization components of the web-deployed sketch-to-3D modeling system as shown in Fig. 1. This approach leverages the power and flexibility of WebGL, a low-level graphics API that allows for hardware-accelerated 3D rendering directly within web browsers, and the comprehensive Three.js library, which provides a high-level, scene graph-based API for building complex 3D scenes and animations [41] [42].

The core of the 3D rendering system is the Three.js scene, which is initialized with a perspective camera, a WebGL renderer, and various lighting components. The scene is then populated with the 3D building models that are reconstructed from the user's sketch input. The system first reads a text file that contains the geometric information extracted from the user's sketch, including the locations and dimensions of various shapes (circles, rectangles, triangles, and custom polygons) that make up the building's walls. Using the information read from the text file, the system constructs the corresponding 3D geometries for each shape and extruded polygons for triangles and custom shapes. Then we employ a CSG library (Three-BVH-CSG) to perform boolean oper-

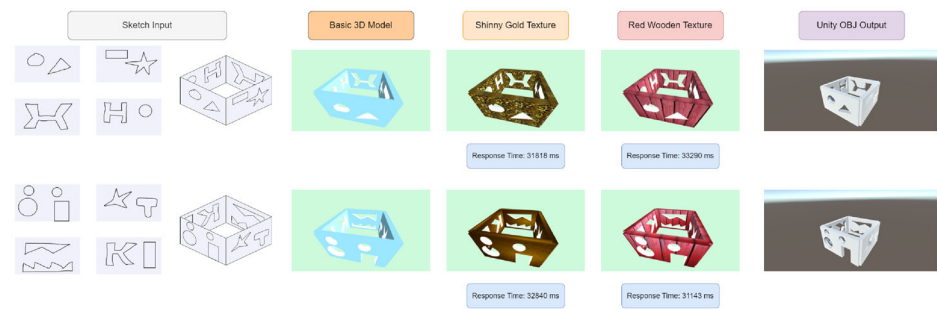
ations on the 3D geometries, effectively "subtracting" the shapes that represent doors, windows, and other architectural features from the base wall geometry. This allows the creation of complex building models with accurate openings and structural elements including windows and columns. We position and orient each wall object model in the 3D scene based on the information provided in the text file, ensuring that the complete building structure is accurately represented. Then the system applies a default material (a Phong material with a specified wall color, used here for simplicity) to the building models. Additionally, the system integrates the ability to load and apply custom textures to the walls, which are generated using a diffusion-based approach and made available through a separate texture management system.

The resulting 3D scene is then rendered using the Three.js WebGL renderer, which takes advantage of the GPU hardware acceleration provided by the user's web browser. The scene can be interactively explored and manipulated using the built-in OrbitControls component of Three.js, allowing users to pan, zoom, and rotate the 3D building model. To enable a seamless user experience, the system includes various user interface (UI) elements and interaction mechanisms, as shown in Fig. 1. This includes a button for triggering the texture generation process, a loading progress indicator, and a drawer interface that displays a history of the generated textures, allowing users to quickly switch between different material styles.

3.4.2 Flask

We implemented the sketch-to-3D modeling system as a web-based application using the Flask framework. This approach ensures cross-platform accessibility via a web

Fig. 7 Results of 2D sketch-to-3D model translation with texturing and unity output



browser. The system processes user sketches, extracts geometric information, and generates high-quality textures for 3D models.

Our web application includes error handling mechanisms to handle exceptions that may occur during the sketch processing and texture generation steps. Appropriate error messages are returned to the client, providing feedback on the success or failure of the requested operations. The web application also manages the necessary directory structure and file operations for handling the input sketches, output models, and generated textures. Additionally, the web application is designed with unit testing in mind. It supports the use of environment variables to enable unit testing mode, which allows for the specific configuration of input and output paths, as well as the generation of test images for validation purposes.

4 Results

Figure 7 showcases the results of our 2D sketch-to-3D model translation system, demonstrating the transformation of various hand-drawn sketches into fully realized 3D models, complete with texturing and export to a Unity environment.

1. **Sketch Input:** The leftmost column displays the initial 2D sketches used as input. These sketches vary in complexity and include basic geometric shapes, as well as more intricate designs.
2. **Basic 3D Model:** The second column illustrates the initial 3D models generated from the 2D sketches using our constructive solid geometry (CSG) based system. These models accurately reflect the structure and layout of the sketches.
3. **Texturing:** The third and fourth columns show the application of different textures to the 3D models. Two types of textures are applied: shiny gold and red wooden textures. These textured models demonstrate the versatility of our system in applying various surface materials to the generated models, enhancing their visual appeal and realism.

4. **Response Time:** Below each textured model, the response time for generating and texturing the model is displayed. This metric provides insight into the efficiency of our system, with times ranging around 31,000 to 33,000 milliseconds.

5. **Unity OBJ Output:** The rightmost column presents the final 3D models as rendered in the Unity game engine. These models have been exported in the OBJ format, showcasing the system's capability to integrate seamlessly with popular game development platforms. The Unity-rendered models retain the structural integrity and details of the original sketches, demonstrating the system's effectiveness in maintaining accuracy through the transformation process.

5 Future work

Further goals of our project are currently to complete the generation of wall decorations and to finish the analysis of sketches of indoor objects with the generation of objects. At present, there are no lightweight 3D generative models suitable for web-based real-time rendering. A suggested way to use 3D generative models is to prepare considerable 3D objects by generative models and store them in the database with labels. Another machine learning model can relate user's input, including sketched shapes and themes, and stored 3D objects. However, customized window/door shapes on walls can be problematic. Another way is to use CSG to generate windows and doors. It's possible to design a special set of 3D geometric combinations with CSG operations to deal with customized shapes.

Additionally, considering the growing popularity of Blender in 3D modeling workflows, exploring ways to integrate Blender-compatible outputs into our system is a promising direction. However, generating Blender models within a web-based real-time rendering context poses technical challenges. A potential solution would be to develop a hybrid framework that exports 3D objects in Blender-compatible formats while maintaining real-time generation capabilities for the web.

Two extra possibilities for future investigation are noted. First, as our present models rely on basic hand-drawn inputs without depth information, the problem of depth ambiguity in sketches continues to be a major obstacle. Creating techniques to deduce depth and structural relationships from user sketches has the potential to greatly improve 3D object generation's accuracy and realism. Second, the incorporation of reference images, as used in techniques such as 3D Gaussian splatting [43], may enhance the representation of intricate object geometries and offer more spatial information. Investigating the ways in which these reference-based methods might enhance sketch-based modeling systems could open up new opportunities for 3D design applications with enhanced fidelity and usability.

6 Conclusion

In this research, we have developed an innovative method for translating 2D sketches into 3D assets using machine learning and Constructive Solid Geometry (CSG) techniques. It is able to output 3D objects that can be utilized in game engines including Unity. Our approach enables users without extensive 3D modeling experience to intuitively create complex structures by addressing the long-standing challenge of translating freehand sketches into 3D models.

We have constructed efficient sketch recognition methods through vertex and edge analysis, which supports customized shapes and maintains precise vertex and position information. Using CSG operations to construct complex 3D geometries from simple primitives also helps maintain the fidelity of the original sketches, and it achieves the computational efficiency that allows real-time rendering. We also incorporated diffusion-based models for generating realistic textures, enhancing the visual quality of the 3D models. The final results are combined in a web-based interface with real-time rendering capabilities for immediate feedback and iterative design.

Our research is seminal to the problem of translating 2D sketches into 3D objects, and, moreover, the further 2D to 3D translation. It has strong extensibility and potential. By continuing to refine and extend it, we can further enhance its utility and impact in various 3D design applications. Also, the implications of this research are far-reaching. It offers significant benefits in fields such as architecture, construction, video game design, and interactive media. By streamlining the process of creating 3D models from sketches, our system enables a wider range of users to engage in 3D design, potentially increasing productivity and fostering creativity.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s00371-024-03758-9>.

Acknowledgements The research receives Dean's Undergraduate Research Fund (DURF) from the College of Arts and Science (CAS) at New York University (NYU).

Author Contributions GK and YC came up with the idea. Both YC and YL wrote the main manuscript text and YC prepared all the figures and visual representations. GK provided technical and administrative consultancy. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Wang, P., Wu, P., Wang, J., Chi, H.-L., Wang, X.: A critical review of the use of virtual reality in construction engineering education and training. *Int. J. Environ. Res. Public Health* **15**, 1204 (2018). <https://doi.org/10.3390/ijerph15061204>
2. Yang, H.B., Johanes, M., Kim, F.C., Bernhard, M., Huang, J.: Architectural Sketch to 3D Model: An Experiment on Simple-Form Houses. *Communications in Computer and Information Science*, vol. 1819. Springer, Cham (2023)
3. Huang, J.: Researchers make it easier to transform 2d sketches into 3D models. *Manufacturing Business Technology* (2018). <https://www.mbtmag.com/home/news/21102143/researchers-make-it-easier-to-transform-2d-sketches-into-3d-models>
4. Pizarro, P.N., Hirschfeld, N., Sipiran, I., Saavedra, J.M.: Automatic floor plan analysis and recognition. *Autom. Constr.* **141**, 104435 (2022)
5. Olsen, L., Samavati, F.F., Sousa, M.C., Jorge, J.A.: Sketch-based modeling: a survey. *Comput. Graph.* **33**(1), 85–103 (2009)
6. Chowdhury, P.N., Bhunia, A.K., Sain, A., Koley, S., Xiang, T., Song, Y.Z.: Democratising 2D sketch to 3D shape retrieval through pivoting. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*
7. Shao, T., Li, W., Zhou, K., Xu, W., Guo, B., Mitra, N.J.: Interpreting concept sketches. *ACM Trans. Graph.* **32**(4), 1–10 (2013)
8. Lin, H., Zhang, R., Xu, C., Wang, H.: Text-to-3D generation with bidirectional diffusion using both 2D and 3D priors. *arXiv preprint arXiv:2401.09822* (2024)
9. Xu, K., Huang, H., Xu, W., Guerrero, P., Mitra, N.J.: Text-to-3D shape generation: a survey. *arXiv preprint arXiv:2403.13289* (2024)
10. Poux, F.: 3D deep learning roadmap to future-proof your career 830 (+19 resources). *Medium* (2024). <https://medium.com/@florentpoux/3d-deep-learningroadmap-to-future-proof-your-career-19-resources-54724baf0744>
11. Google: Quick, draw! The data. <https://quickdraw.withgoogle.com/data>. Last accessed 20 Apr 2024
12. Eitz, M., Hays, J., Alexa, M.: How do humans sketch objects? *ACM Trans. Graph.* **31**, 4 (2012). <https://doi.org/10.1145/2185520.2185540>
13. Robert, F.: Hand-drawn shapes dataset. GitHub repository (2023). <https://github.com/frobertpoxto/hand-drawn-shapes-dataset>
14. Sangkloy, P., Burnell, N., Ham, C., Hays, J.: The sketchy database: learning to retrieve badly drawn bunnies. *ACM Trans. Graph.* **35**, 4, Article 119 (2016). <https://doi.org/10.1145/2897824.2925954>

15. Li, M., Lin, Z., Mech, R., Yumer, E., Ramanan, D.: Photo-sketching: inferring contour drawings from images. In: WACV (2019). <https://arxiv.org/abs/1901.00542>
16. Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. In: Proceedings of the Advances in Neural Information Processing Systems (2015)
17. Luo, Y., Yu, T., Zheng, J., Ding, Y.: Design of engineering drawing recognition system based on Yolo V4. In: 2022 IEEE 6th Information Technology and Mechatronics Engineering Conference 2022, pp. 1221–1225. <https://doi.org/10.1109/ITOEC53115.2022.9734453>
18. Yu, Q., Yang, Y., Liu, F., et al.: Sketch-a-net: a deep neural network that beats humans. *Int. J. Comput. Vis.* **122**, 411–425 (2017). <https://doi.org/10.1007/s11263-016-0932-3>
19. Muhammad, U.R., Yang, Y., Song, Y., Xiang, T., Hospedales, T.M.: Learning deep sketch abstraction. In: Computer Vision and Pattern Recognition (CVPR) (2018)
20. Manfredi, G., Capece, N., Erra, U., Gruosso, M.: TreeSketchNet: from sketch to 3D tree parameters generation. *ACM Trans. Intell. Syst. Technol.* **14**(3), 1–29 (2023)
21. Xu, P., Hospedales, T.M., Yin, Q., Song, Y.-Z., Xiang, T., Wang, L.: Deep learning for free-hand sketch: a survey. *arXiv*, [arXiv:2001.02600](https://arxiv.org/abs/2001.02600). <https://arxiv.org/abs/2001.02600> (2022)
22. Sangkloy, P., Burnell, N., Ham, C., Hays, J.: The sketchy database: learning to retrieve badly drawn bunnies. *ACM Trans. Graph.* **35**(4), 1–12 (2016)
23. Eitz, M., Richter, R., Boubekur, T., Hildebrand, K., Alexa, M.: Sketch-based shape retrieval. *ACM Trans. Graph.* **31**(4), 1–10 (2012)
24. Venkataramaiyer, R.B.: Geometric understanding of sketches. In: *arXiv preprint* [arXiv:2204.06675](https://arxiv.org/abs/2204.06675) (2022)
25. Nishida, G., Garcia-Dorado, I., Aliaga, D.G., Benes, B., Bousseau, A.: Interactive sketching of urban procedural models. *ACM Trans. Graph.* **35**(4), 1–11 (2016)
26. Fayolle, P.A., Friedrich, M.: A survey of methods for converting unstructured data to CSG Models. *Comput. Aided Des.* **168**, 103655 (2024). <https://doi.org/10.1016/j.cad.2023.103655>
27. Wang, J., Lin, J., Yu, Q., Liu, R., Chen, Y., Yu, S.X.: 3D shape reconstruction from free-hand sketches. In: Karlinsky, L., Michaeli, T., Nishino, K. (eds.) *Computer Vision: ECCV 2022 Workshops. ECCV 2022. Lecture Notes in Computer Science*, vol. 13808. Springer, Cham. https://doi.org/10.1007/978-3-031-25085-9_11
28. Deng, K., Yang, G., Ramanan, D., Zhu, J.Y.: 3D-aware conditional image synthesis. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2023)
29. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. [arXiv:2006.11239](https://arxiv.org/abs/2006.11239) [cs.LG] (2020)
30. Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial networks. In: *Advances in Neural Information Processing Systems (NIPS)*, pp. 2672–2680 (2014)
31. Kingma, D.P., Welling, M.: Auto-encoding variational bayes. In: *International Conference on Learning Representations (ICLR)* (2014)
32. Dhariwal, P., Nichol, A.: Diffusion models beat GANs on image synthesis. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **34**, 8780–8794 (2021)
33. Nichol, A., Dhariwal, P.: Improved denoising diffusion probabilistic models. In: *International Conference on Machine Learning (ICML)*, pp. 8162–8171 (2021)
34. Tanaka, M., Asano, T., Higashino, C.: Isometric conversion of mechanical sketches into 3D models. *Comput. Aided Des. Appl* **18**, 722–785 (2020). <https://doi.org/10.14733/cadaps.2021.772-785>
35. Tanaka, M., Terano, M., Asano, T., Higashino, C.: Method to automatically convert sketches of mechanical objects into 3D models. Okayama University of Science, Aikoku Alpha Corporation. *Computer-Aided Design and Applications* (2020). <https://doi.org/10.14733/cadaps.2020.1168-1176>
36. Wang, J., Lin, J., Yu, Q., Liu, R., Chen, Y., Yu, S.X.: 3D shape reconstruction from free-hand sketches. In: Karlinsky, L., Michaeli, T., Nishino, K. (eds.) *Computer Vision: ECCV 2022 Workshops. ECCV 2022. Lecture Notes in Computer Science*, vol. 13808. Springer, Cham. https://doi.org/10.1007/978-3-031-25085-9_11
37. Barron, J.T., Malik, J.: Shape, illumination, and reflectance from shading. *IEEE Trans. Pattern Anal. Mach. Intell.* **37**(8), 1670–1687 (2015). <https://doi.org/10.1109/TPAMI.2014.2377712>
38. Plumed, R., Company, P., Varley, P.A.C., Martin, R.R.: From sketches to CAM models: perceiving pockets and steps in single-view wireframe sketches of polyhedral shapes. In: *Proceedings of the 2013 ACM Conference on Pervasive and Ubiquitous Computing Adjunct Publication* (2013), 951–958. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2494091.2499207>
39. Plumed, R., Varley, P.A.C., Company, P., Martin, R.: Extracting datums to reconstruct CSG models from 2D engineering sketches of polyhedral shapes. *Comput. Graph.* **102**, 349–359 (2022)
40. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695 (2022)
41. Khronos Group.: WebGL 2.0 Specification. <https://registry.khronos.org/webgl/specs/latest/2.0/>
42. Cabello, R., et al.: Three.js - JavaScript 3D Library. *Computer Graphics & Visualization* (2024). <https://threejs.org/docs/>
43. Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G.: 3D Gaussian splatting for real-time radiance field rendering. *arXiv preprint* [arXiv:2308.04079](https://arxiv.org/abs/2308.04079) (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Yiming Chen received his dual B.A. degrees in Mathematics and Computer Science from New York University in 2024. He is currently pursuing a Master's degree in Computer Science at the Georgia Institute of Technology, expected in 2026. His research focuses on machine learning, computer vision, and diffusion-based concept generation.



Yihang Liu received his dual B.A. degrees in Mathematics and Computer Science from New York University in 2024. He is currently a graduate student at the University of Pennsylvania, pursuing a Master's degree in Robotics. His research interests include computer vision, generative models, and robotics.



Gizem Kayar-Ceylan received her Ph.D. in Computer Science in 2014 from the Albert Ludwig Freiburg University. She is currently a Clinical Associate Professor with the Department of Computer Science, New York University, and continues to have been supervising various student research projects for years. Her research interest includes conventional and ML-based physically based simulation techniques and also use of advanced graphics techniques in video games.